Engineering a Formal, Executable x86 ISA Simulator for Software Verification

Shilpi Goel, Warren A. Hunt Jr. and Matt Kaufmann

Abstract Construction of a formal model of a computing system is a necessary practice in formal verification. The results of formal analysis can only be valued to the same degree as the model itself. Model development is error-prone, not only due to the complexity of the system being modeled, but also because it involves addressing disparate requirements. For example, a formal model should be defined using simple constructs to enable efficient reasoning but it should also be optimized to offer fast concrete simulations. Models of large computing systems are themselves large software systems and must be subject to rigorous validation. We describe our formal, executable model of the x86 instruction-set architecture; we use our model to reason about x86 machine-code programs. Validation of our x86 ISA model is done by co-simulating it regularly against a physical x86 machine. We present design decisions made during model development to optimize both validation and verification, i.e., efficiency of both simulation and reasoning. Our engineering process provides insight into the development of a software verification and model animation framework from the points of view of accuracy, efficiency, scalability, maintainability, and usability.

1 Introduction

The development of large computing systems is almost always preceded by a modeling and analysis stage. When building a complex computing system, success is often determined by the accuracy of its model and the quality of analysis this model receives. For example, it is now impossible to design and manufacture a micro-

S. Goel (✉) · W.A. Hunt Jr. · M. Kaufmann
Department of Computer Science, The University of Texas at Austin,
2317 Speedway, M/S D9500, Austin, TX, USA
e-mail: shigoel@cs.utexas.edu

W.A. Hunt Jr.
e-mail: hunt@cs.utexas.edu

M. Kaufmann
e-mail: kaufmann@cs.utexas.edu

© Springer International Publishing AG 2017

M.G. Hinchey et al. (eds.), *Provably Correct Systems*, NASA Monographs
in Systems and Software Engineering, DOI 10.1007/978-3-319-48628-4_8

processor chip without extensive modeling, simulation, and product validation. The rising complexity and use of software systems necessitates building scalable tools for analyzing program behavior. To this end, we have used a rigorous mathematical approach to specify the x86 instruction-set architecture (ISA) for the formal analysis of x86 machine-code programs. Our specification of the x86 ISA is regularly validated to increase trust in the applicability of the results of formal analysis.

In principle, our approach to specifying an instruction-set architecture was known by von Neumann [1], Turing [2], and Zuse [3]. Our x86 ISA specification is simply a transition function that transforms one ISA-level state into the next one; this operation can be repeated to simulate any x86 program. A critical factor that makes the x86 ISA difficult to specify is its sheer size and complexity. The Intel manuals [4] that describe the x86 ISA are large documents consisting of around 3700 p. The x86 architecture continues to evolve. In fact, since we began our project in 2010, the x86 ISA has been significantly extended [5] — the AVX (Advanced Vector Extensions) is one such extension. This continued evolution is common for industrial products, which are modified and extended as business requirements demand.

In light of the size and complexity of the x86 ISA, the system chosen for its specification plays an important role. This system should be capable of rigorously specifying an ISA as large as the x86, and this specification should scale well and be maintainable. This system should offer an environment for both formal reasoning and validation; the system should provide a mathematical logic to support formal analysis about both the model itself and the x86 programs simulated on it, and the system's logic should be executable so that the ISA specification can be validated by performing *co-simulations* (i.e., comparisons of simulations of x86 programs on the model with corresponding executions on a real x86 processor). We use the ACL2 theorem-proving system [6, 7] to model the x86 ISA. ACL2 is a state-of-the-art system used in industry, and we have previously used ACL2 and its predecessor NQTHM [8] to model microprocessors at the RTL and ISA levels [9, 10].

Our goal is to ensure software reliability, and constructing a formal model of the x86 ISA is simply a means to that end. However, engineering a model of this scale is hardly a trivial undertaking. The model-building process is inherently error-prone because of the size and complexity of the ISA, and it is complicated further by the nature of demands placed on it. For example, to facilitate faster co-simulations, the model must provide high execution efficiency, and to support scalable formal analysis, the model must be defined using simple constructs that are easy to reason about. There is a tension between these two demands — optimizing a function for execution efficiency often increases the complexity of its underlying algorithm, making it more difficult to reason about the function. Another example of opposing demands, typical of large software projects, is completeness versus maintainability. Extending the model by supporting more x86 features and instructions should not make it unwieldy and difficult to maintain. Every design decision taken while building the model has to factor in such inherently disparate requirements. In this chapter, instead of focusing on our work on program verification, we discuss the framework that we built to attain our goal. We present our formal, executable model of the x86 ISA from an engineering standpoint and describe our design decisions, big and small, in detail.

Our x86 ISA model is more than 45,000 lines of executable ACL2 logic, with around 20,000 lines more for our libraries that aid in co-simulations and formal analysis. Our current focus is on the 64-bit mode[1] of the x86 ISA; in the rest of this chapter, whenever we refer to our x86 model, we mean our ISA model of the x86 processor in this 64-bit mode. The simulation speed of our model is either around 330,000 instructions/second or 3.3 million instructions/second, depending on its mode of operation.[2] To our knowledge, this is the fastest formal simulator for this complex an ISA model. We currently have a specification of 407 machine opcodes that include supervisor-mode and floating-point instructions. We have specified access control (via privileges and permissions) and address translation by capturing IA-32e paging and segmentation. We model the single-processor x86 architecture, but we someday hope to have a multi-core version. Just as the x86 ISA has been evolving since its introduction by Intel, our model continues to evolve as we support more features and instructions. Our x86 ISA model and related libraries for formal analysis and simulation are freely available [12] with a permissive BSD license. Their documentation [13] is available online too.

This presentation of our evolving x86 model is meant to be self-contained. We provide relevant details about ACL2 and the x86 ISA when needed. The reader can start with only the notion that a state-machine transformer will be presented, one which can be used both for simulation and for reasoning.

We begin in Sect. 2 by describing our modeling approach. The next two Sects. 3 and 4 present in some detail our model and its validation. After briefly describing in Sect. 5 the use of our model for software verification, we close in Sects. 6 and 7 with related work and concluding remarks.

2 Approach

We take a three-pronged approach to the construction of a model to be used for software verification. In Sect. 2.1, we explain our decision to use an interactive theorem prover as the choice of formal tool. Section 2.2 discusses why we perform analysis at the level of machine code to ensure software reliability. Having discussed the tool and the target of analysis, Sect. 2.3 sets out design goals for our framework, which includes a unified model for both simulation and formal analysis.

2.1 Formal Methodology

Our aim is to develop capabilities for thorough analysis of software to ascertain if, under a given set of conditions, it behaves as expected during run-time. Even

[1] Intel's IA-32e mode [11] of operation offers two sub-modes: the compatibility mode (which is similar to 32-bit protected mode) and 64-bit mode.

[2] The simulation speed is discussed in Sect. 4.

trivial programs rely on many complex conditions and properties for their correct execution. Such properties demand formal scrutiny, simply due to their complexity. Some examples are:

- The stack or the heap should not overwrite the program in any execution.
- The operating system kernel data should be invisible to application programs.
- Segmentation and paging data-structures should be set up correctly by the OS kernel. This is a property that is critical to establishing many others; e.g., a correct configuration will help in determining if process isolation is assured.

The thorough analysis of software requires capturing the *intent*, including the preconditions and expected behavior of the program. Fully automatic program analysis tools, like static and dynamic analyzers, do not impose this requirement of writing specifications on the user, but as a result, they are narrow in their scope. Though successful in carrying out their particular functions, like detecting memory safety violations, these tools do not support even the statement of many complex properties. In addition to such properties, useful metrics like program memory and space usage are difficult, if not impossible, to obtain using these tools.

The development of the CLI stack [14] in the 1980s using NQTHM, a Boyer–Moore theorem prover, has already demonstrated that theorem proving techniques alone can be applied to obtain various kinds of guarantees about computing systems. We will utilize the state-of-the-art in formal verification by coupling interactive theorem proving with automated tools, for example SAT solvers, to facilitate the development and delivery of secure systems.

2.1.1 ACL2

Our theorem prover of choice is the ACL2 system [6, 7], a descendant of the Boyer–Moore theorem prover used in the CLI stack project. It is a mathematical logic (a first-order logic of recursive functions) based on an applicative subset of Common Lisp as well as a mechanical theorem prover used to prove theorems in that logic. ACL2 is also a programming language, which offers the execution efficiency provided by underlying Lisp compilers, thereby enabling the construction of efficient executable formal models.

The ACL2 theorem prover has many automated proof strategies, and the application of a proof strategy recursively decomposes goals until all subgoals are proved. Users can extend the prover by adding their own proof strategies and external deduction tools, like SAT/SMT solvers, by using *clause processors* [15]. We often use GL [16], a framework for proving theorems involving finite objects using either a formally verified BDD utility or an external SAT solver like Glucose [17, 18].

ACL2 and its libraries, called *books*, are freely available [19] and include extensive documentation, with over 10,000 topics [20, 21]. These books, developed over many years, are typically included in ACL2 projects to build on existing definitions and theorems. Books can be certified by ACL2 to ensure their soundness.

ACL2's authors are the recipients of 2005 ACM Software System Award [22]. ACL2 is an industrial-strength system; it is regularly used in both academic and commercial applications [23]. For example, Centaur Technology [24], an x86-compatible processor vendor, uses ACL2 every day for modeling and verifying Centaur processor designs [25–27]. Other organizations with significant usage of ACL2 for formal verification are AMD [28–31], IBM [32–35], Intel [36], Kestrel [37, 38], Oracle [39], and Rockwell-Collins [40–42].

We now briefly discuss some ACL2/Lisp features that are especially important when building large formal and executable models. Some efficient data structures provided by ACL2 are discussed in Sects. 3.1 and 3.2.

Guards Though Lisp is syntactically untyped, the Common Lisp standard [43] specifies an intended domain for each Lisp primitive. The behavior of those primitives outside these domains depends on the particular Lisp implementation.

In ACL2, a mechanism called *guards* [44] is used to specify the intended domain of a function. *Completion axioms* define the behavior of a primitive for every possible input; thus, all ACL2 functions are logically total. When the guards of a function `f` are verified, it means that `f` respects the guards of all functions used in its body. A guard-verified function can thus be called natively in the host Lisp (as opposed to evaluation within ACL2), such that every ensuing call of any function `g` respects the guard of `g`. Thus, verification of the guards of a function improves its execution speed. Guards have no effect during reasoning — they do not affect the logical axiom that is added when an ACL2 function is defined.

Another feature that can allow faster execution of ACL2 functions is `mbe` [45] ("must be equal"). This construct allows the following form to be used in functions:

```
(mbe :logic <logic-code> :exec <exec-code>)
```

Users can choose to write simple code suitable for efficient reasoning in the `:logic` part and code optimized for execution efficiency in the `:exec` part. Logically, this `mbe` form is equal to `<logic-code>`. During execution, this form is equal to `<exec-code>` if the function's guards are verified. The `mbe` form generates proof obligations that are added to the guard proof obligations; when proved, they establish the equality of `<logic-code>` and `<exec-code>`, thereby allowing `<exec-code>` to be executed in place of `<logic-code>` when the function can be safely run natively in the host Lisp.

Type Declarations Common Lisp supports arbitrary-precision arithmetic; it imposes no limit on the magnitude of a number [46]. Machine integers, called *fixnums*, are efficiently represented and fast built-in arithmetic operations can be used when working with them. Integers with an unlimited number of bits are called *bignums* and special arithmetic functions are required when working with them. If an integer gets too big (where the "bigness" is defined by the Common Lisp implementation), Lisp promotes it from a fixnum to a bignum automatically and subsequent operations involving these numbers are the slower bignum operations. Therefore, avoiding bignum operations when possible is important for simulation efficiency.

Annotating ACL2/Lisp code with *type declarations* can avoid run-time type checking. Type declarations are a Common Lisp mechanism that allows the user to transmit information about types of objects to the underlying Lisp compiler. E.g., the user might declare that a certain variable, say `x`, is always of type `(unsigned-byte 32)`, i.e., an unsigned integer of width 32 bits. If the Lisp implementation's most positive fixnum is greater than or equal to $2^{32} - 1$ (true for 64-bit Lisp implementations), this type declaration enables the compiler to generate efficient machine code that does not involve either bignum computations or run-time type-checks.

In ACL2, type declarations need to be justified. They are added to the guard proof obligation to ensure that the execution of the function with type declarations in ACL2 can be safely done in the host Lisp without the possibility of encountering any run-time type violations.

Other Efficiency Concerns Two other efficiency-related issues need to be mentioned here — avoid consing (creation of ordered-pair data structures) and use inlining. Consing is a potentially expensive operation that allocates memory on the heap; avoiding it can substantially increase ACL2/Lisp code's execution efficiency.

Defining many small functions, rather than a single large function, is preferred for reasoning because it decomposes functionality into small easy-to-understand abstractions. However, for execution, function call overhead is high. Inlining small functions can improve the code's execution performance, but inlining big functions can cause code bloat. The decision to inline a function must be made judiciously.

Constrained Functions ACL2 provides a mechanism called `encapsulate` to introduce *constrained or uninterpreted functions*. The only things known about a constrained function are its name, signature (arity), and constraints, if any. Constrained functions can be introduced together with logical properties of those functions; but unlike defined functions, they are non-executable.

Trust Tags ACL2 allows users to extend the prover in ways otherwise not allowed by introducing a *trust tag* [47]. Typically, users extend the prover by creating ACL2 books that perform a potentially dangerous evaluation. An example of a dangerous evaluation is a function `foo` that invokes the command `date` on the underlying OS and returns the output; this is not a function in the logical sense because it may return different outputs for the same input (and in sound logics, `foo() == foo()` is always true). Books containing functions like `foo` must be certified with a trust tag. Note that careful use of `foo` (e.g., in a way that does not interfere with reasoning) should still be safe. If a book uses a trust tag, then it means that the functionality provided by the book is *trusted*; it is acknowledged that the formulas proved using that book are valid only if the book's functionality is correct. Among its other uses, trust tags allow the integration of various external tools [15] with ACL2.

Untouchable Functions ACL2 functions can be made *untouchable* [48] — it is syntactically illegal to call an untouchable function directly. Typically, this is done to preserve abstractions; e.g., if a top-level function calls several inferior functions, but

the top-level function needs to be perceived as an atomic operation, then the inferior functions are made untouchable.

2.2 Machine-Code Analysis

Even though using high-level semantics for program verification may seem appealing, we advocate performing software verification at the machine-code level. In the case where we only have a machine-code program (e.g., an executable downloaded from the Internet), direct source-level analysis is impossible. Even when high-level source code is available for analysis, the implementation of the programming language needs to be trusted (or verified); for example, the memory semantics being offered by the programming language are eventually provided by the underlying memory system. Moreover, analysis of high-level programs does not account for the possibility of bugs introduced by compilers, thereby decreasing confidence that properties proved during analysis will hold at run-time. Even with the advent of practical and verified mainstream compilers [49], assurances that a program does not access unauthorized memory regions cannot be obtained unless low-level operating system code, on which programs rely for various services [50–53], is also analyzed; often, such low-level OS code is written in assembly language. Therefore, it is prudent to analyze machine code for modern commercially available processors. A big benefit of machine-code analysis tools is their universal applicability; they can be used to verify all programs that can compile down to the supported hardware platform. Also, similar to the CLI stack, machine-code analysis can be used to prove the programming language implementation on a given processor correct; software verification can then proceed at the programming language level.

Our focus is on the 64-bit mode of the x86 ISA. There are two main reasons for choosing the x86 ISA over other available ISAs.

1. Since x86 is the dominant processor architecture for non-embedded devices, a simulation and verification framework based on the x86 ISA will find immediate practical application. Also, most modern non-embedded devices run in 64-bit mode.
2 The x86 ISA is one of the most complicated processor architectures, and hence, any success in this project will demonstrate that our formal methods technology is mature enough to handle real-world, industrial problems.

In spite of the benefits of machine-code analysis, formal verification of machine code has few practitioners [54–56], though it is steadily gaining ground. Information contained in the abstractions provided by high-level languages, like the layout or "shape" of data structures, is mostly lost at the machine-code level, which makes machine-code analysis detail-intensive and tedious. Work has been done to provably lift low-level verification to a higher-level reasoning process [57–59]. However, in this chapter, our focus is not on verification techniques themselves but on the engineering

decisions involved in just constructing a reliable model of the x86 ISA to enable the application of those techniques.

2.3 Design Goals

Inspection of the behavior of machine code is often done by employing instruction-set simulators [60–62], which use sophisticated techniques to achieve high-speed simulation of machine instructions and are written in efficient programming languages like C and C++. Even though such simulators are commonly used as ISA reference models, they are not formal specifications of these ISAs and hence, do not directly provide an infrastructure for formal code analysis.

ISA models written in formal specification languages allow the direct application of formal reasoning tools. Verification using these formal models indeed increases confidence in the reliability of machine code, but this analysis can not be fully trusted until the model is known to be faithful to the processor. One way to increase confidence in the accuracy of a model is by performing co-simulations to validate it against a real machine. Co-simulation is the process of executing a machine program on the processor as well as on the model; if the state of the processor and the model are the same after every instruction, then the model is faithful to the processor for at least those instructions (and data). Unfortunately, formal specification languages do not usually provide an efficient execution environment. Model validation can also be done by performing extensive code reviews — an approach that is independent of the model's execution efficiency. However, code reviews are time and labor intensive. The reviewers would need to study the vendors' ISA manuals, which are long documents mostly consisting of prose [4, 63]. There is always the danger of making subjective judgments about the processor's behavior, especially if the processor has a complicated ISA and if some features are under-specified. Therefore, code reviews can supplement co-simulations, but not supplant them.

This comparison of ISA simulators and formal ISA specifications justifies the need to develop a formal ISA model whose execution speed allows efficient co-simulations for model validation. A validated model increases trust in the applicability of the results of formal verification. Another benefit of such a unified model is that it reduces the overhead associated with designing, developing, and maintaining two separate models — in particular, maintaining them consistently. ACL2 is an excellent fit for this task; it is both a mathematical logic and a programming language.

We considered the following factors over the course of engineering our model:

Accuracy No simplification in the semantics of the ISA may be done. For example, a machine model should be based on machine integers, not unbounded integers.

Efficiency The benefits of a unified model for simulation and formal verification come at a price: we must mitigate the trade-off between simulation and reasoning efficiency. If the model is defined using simple constructs, it enables easier reasoning but can offer poor execution performance. Definitions optimized for execution

efficiency are typically more difficult to reason with because they may use an algorithm different from the "obvious" one. Abstraction techniques (e.g., those provided by ACL2 features like `mbe` and abstract stobjs [64]) provide leverage to build an efficient model for both reasoning and execution.

Usability In its role as a simulator, the model must come with dynamic instrumentation tools, similar to the GNU Debugger (GDB) and Intel's Pin Tool [65]. Such tools facilitate examination of the machine state during concrete simulations of programs, thereby helping in model and program debugging. In its role as a reasoning framework, the model must be accompanied with libraries that aid in machine-code analysis. For ease of use, the model is documented (available online [13]) — from the point of view of a simulator as well as a reasoning framework.

The complexity of the x86 ISA model will increase as more features are added to it, and this added complexity will inevitably make reasoning more involved. Balancing *verification effort* and *verification utility* is a highly pertinent issue. For example, users might not want to reason about an application program at the level of physical memory, i.e., taking into account address translations and access rights management. This is because it is customary for application programs not to have direct access to the system data structures.[3] The memory model seen by application programs is that of linear memory, which is an OS-constructed abstraction of the complicated underlying memory management mechanisms that are based on physical memory. Therefore, verification of application programs can be performed at the level of linear memory, if the OS routines that manage the linear memory abstraction can be either trusted or proved correct. However, system programs, like kernel routines, must necessarily have a view of physical memory since these programs can access/modify system data structures. In light of the above, the x86 ISA model should provide the option to deactivate some ISA features, enabling the user to do varying depths of analysis, depending on the kind of programs being considered for verification. Consequently, our x86 model has two modes of operation: the *programmer-level mode* for the analysis of application programs and the *system-level mode* for the analysis of system programs. Details about these modes are in Sect. 3.3.

Usability is further supported by the free availability of our model, under a BSD 3-Clause license, from the Github webpage of the ACL2 Community books [12]. This permissive license can facilitate the adoption of our model.

Scalability and Maintainability The model of the x86 ISA will be large and it will keep growing larger, due to complexity of the ISA and addition of new features. Thus, the maintainability of the model is an important factor influencing its design decisions; the two modes of operation of the model help in this regard.

[3] The access rights of an application program are ultimately governed by the OS, but it is extremely rare (and inadvisable) for an application program to have direct access to system resources.

3 Model Definition

Our ACL2-based formal and executable model [66] of the x86 ISA has a specification of all addressing modes, most user-level instructions (including floating point instructions), some system-level instructions, paging, and segmentation. The instructions that are unspecified, as of this writing, include those that access co-processor I/O address spaces, rarely used BCD instructions, cache-related instructions and virtual machine extensions.

We use the Intel manuals [4] as reference documents for model development. Ambiguities are resolved by cross-referencing AMD manuals [63], running tests on real processors, and consulting with x86 processor architects.

Our x86 ISA model provides an interpreter-style operational semantics [67], where the meaning of a machine-code program is given by a recursively-defined interpreter over the machine-code language's syntax and the processor's state. Broadly, there are four main aspects to a model defined using an interpreter-style operational semantics; we describe them using the x86 ISA.

1. **State**: Components of the x86 ISA state currently supported by our model are described in Table 1. Sections 3.1–3.3 describe how the state is defined and accessed in our x86 model.
2. **Instruction Semantic Functions**: Each machine instruction is specified by a function that defines its semantics. This semantic function takes an initial x86 state as input and returns an appropriately modified next state as output. Section 3.4 describes the instruction semantic functions in our model.
3. **Step and Run Functions**: A step function fetches, decodes, and executes a machine instruction by calling its associated instruction semantic function. A run function takes the number `n` of instructions to be executed and an initial x86 state. The run function function either takes `n` steps or terminates early if an unrecoverable error is encountered, returning an appropriately modified final x86 state in either case. Section 3.5 describes the step and run functions in our model.

The semantics of an x86 machine-code program are given by the composition of the semantics of its constituent instructions, and we can use ACL2 both to simulate concrete program runs and to reason about symbolic program runs.

Our x86 model is layered to facilitate ease of maintenance and scalability. We describe each layer below and discuss the reasons for our modeling decisions.

3.1 Concrete State

The x86 state is defined using an ACL2 data structure called a *concrete stobj* [68]. "Stobj" stands for "Single-Threaded Object"; such a structure is a mutable object with applicative semantics. Stobjs offer high execution performance by allowing destructive assignments while providing copy-on-write semantics for reasoning. Consider the `defstobj` event below that introduces a stobj called `foo`.

Table 1 Components of the x86 processor that our currently supported by our model

#	Component	Type and size	Comments
1	General-Purpose Registers	16 64-bit wide registers	`rax`, `rcx`, `r11`, etc.
2	Instruction Pointer	1 64-bit wide register	`rip` register
3	Flags Register	1 64-bit wide register	`rflags` register
4	Segment Registers	6 16-bit wide registers	`cs`, `ss`, etc.
5	Segmented Memory Management Registers	2 80-bit wide registers	Global and local descriptor registers, `gdtr` and `ldtr`
6	Interrupt and Task Management Registers	2 16-bit wide registers	Interrupt descriptor and task registers, `idtr` and `tr`
7	Control Registers	16 64-bit wide registers	`cr0` to `cr15`
8	Floating-Point Data Registers[a]	8 80-bit wide registers	`fp-data0` to `fp-data7`
9	Floating-Point Control Register	1 16-bit wide register	`fp-ctrl`
10	Floating-Point Status Register	1 16-bit wide register	`fp-status`
11	Floating-Point Tag Register	1 16-bit wide register	`fp-tag`
12	Floating-Point Last Instruction Pointer	1 48-bit wide register	`fp-last-inst`
13	Floating-Point Last Data Pointer	1 48-bit wide register	`fp-last-data`
14	Floating-Point Opcode	1 11-bit wide register	`fp-opcode`
15	XMM Registers	16 128-bit wide registers	`xmm0` to `xmm15`
16	MXCSR Control and Status Register	1 32-bit wide register	`mxcsr`
17	Machine-Specific Registers	6[b] 64-bit wide registers	
18	Byte-Addressable Memory	Models 2^{52} bytes[c]	Main memory

[a]The MMX registers are aliased to the low 64 bits of the FPU's data registers, as dictated by the ISA

[b]Intel defines a lot more than 6 model-specific registers. Our model currently supports 6 of them: `ia32_efer`, `ia32_fs_base`, `ia32_gs_base`, `ia32_kernel_gs_base`, `ia32_lstar`, `ia32_star`, `ia32_fmask`

[c]2^{52} bytes is the largest physical address space provided by modern x86 implementations

```
(defstobj foo
  (a :type (array (signed-byte 64) (2)) :initially 0)
  (b :type (unsigned-byte 48) :initially 10))
```

In logic, the stobj `foo` is simply a list of two elements; the first element (referenced by the field `a`) is a list of two elements, each of which is a signed integer of width 64 and has an initial value of 0, and the second element (referenced by the field `b`) is an unsigned integer of width 48 and has an initial value of 10. For execution, ACL2 enforces sequencing updates to `foo` by way of syntactic restrictions ensuring that only one instance of `foo` exists at all times, thereby allowing destructive assignments. Apart from introducing a stobj, a `defstobj` event also introduces some native functions: recognizers (functions that check whether the stobj and its constituent fields are of the right logical representation), a creator (a function that creates an initial logical representation of the stobj), accessors (functions that read a stobj field), and updaters (functions that write to a stobj field).

We have modeled components listed in Table 1 as fields in the x86 concrete stobj. For components like the segment registers, task register (`tr`), and local descriptor table register (`ldtr`), our model of the state also includes the hidden or shadow parts of these registers, as described in the processor manuals. Some fields in our x86 state are an artifact of the model itself — they do not exist on a real x86 processor. These fields store information about the model state (`ms`), mode of operation (`programmer-level-mode`), environment (`env`), seed for undefined values (`undef`), and information about the operating system (`os-info`). The `ms` field contains information about problems (if any) encountered during a program run. These problems include those arising due to features that are currently unimplemented in our model. The processor state is expected to be correct if `ms` is empty; otherwise, the run function terminates and processor execution is halted. Other fields will be discussed when relevant in the rest of this chapter.

Though the use of concrete stobjs to define the x86 state helps in achieving high simulation efficiency, we can get better performance by choosing the types of the stobj fields judiciously. The stobj fields should be defined in a way that avoids bignum creation when possible. One common technique we employ for this purpose is illustrated by our modeling of the general-purpose registers (GPRs), which are 64-bits wide on an x86 processor. On most contemporary 64-bit Lisp implementations, 64-bit numbers are of type bignum. The GPRs are defined as an array of 16 elements, each of which is a *signed* integer of width 64 bits instead of an *unsigned* 64-bit integer. This approach provides a substantial performance boost because big positive numbers can be stored as small negative numbers; e.g., bignum $2^{64} - 1$ can be represented by -1 when stored as a 64-bit signed integer. Similar optimizations have been made for other fields in the x86 state.

The byte-addressable memory deserves special mention. We support a memory of size 2^{52} bytes (i.e., 4096TB), the largest provided by modern x86 implementations. However, allocating 4096TB all at once is impractical, if not impossible. We have implemented a time- and space-efficient memory model [69] in order to keep the model's memory footprint manageable. Our memory model can be viewed as a flat

array consisting of 128MB blocks; memory is allocated on demand in these blocks of 128MB instead of all at once. Three fields in the concrete stobj are used to implement this on-demand memory, `mem-table$c`, `mem-array-next-addr$c`, and `mem-array$c`. `mem-table$c` stores the 25-bit addresses of the 128MB blocks. `mem-array-next-addr$c` stores the 25-bit block address to be allocated next, and `mem-array$c` stores the bytes. Hence, the 52-bit physical address of a byte can be thought of as composed of two parts: the most significant 25 bits are the address of a block and the remaining 27 bits are the address of the offset within that block. The following pseudo-code describes how to compute `memArrayIndex`, the index into `mem-array$c` where the byte corresponding to a physical address `addr` is located; the notation `addr[to:from]` represents the slice of `addr` from bit position `to` to bit position `from`, inclusive of both indices, and `x86$c` represents the concrete x86 state.

```
blockAddr := mem-array(addr[51:27], x86$c)
if valid-p(blockAddr) then
         // Address located in a block previously allocated.
        memArrayIndex := (blockAddr <<  27) | addr[26:0]
else     // A new block needs to be allocated.
        addNewBlock(x86$c)
        memArrayIndex := (mem-array-next-addr << 27) | addr[26:0]
        mem-array-next-addr := mem-array-next-addr + 1
endif
```

With this notion of on-demand memory in our model, a well-formed x86 state needs to have a well-formed memory `good-memp`, i.e., the relationship among the three memory fields should give a correct model of a 4096TB byte-addressable memory. We have proved that this is an invariant of our model. Note that this is a property *about* our model — it assures us that our memory model, despite heavy optimizations, is indeed the one we intended to build.

We access and update a byte in the memory using functions called `mem$ci` and `!mem$ci`. These functions locate a byte using the relationship among the concrete state fields `mem-array$c`, `mem-array-next-addr$c`, and `mem-table$c` described above. We never access or update these three fields directly.

3.2 Abstract State

Though concrete stobjs enable reasoning and efficient simulation, using them has the following drawbacks in the context of our x86 model.

Issue (1) Large logical representation of the x86 state: Logically, the concrete x86 state is a list of lists and integers. Two of the fields that implement the memory model are extremely large linear lists; `mem-table$c` is a list of 2^{25} elements and `mem-array$c` is a list of 2^{28} elements (corresponding to an initial memory allocation of 256MB), which increases if more memory is requested. The size of these lists adversely affects reasoning efficiency — they have to be created in order

to symbolically simulate functions (in logic, using the bit-blasting tool GL mentioned above) that take the x86 state as input.

Issue (2) Expensive guard checking: The recognizer of a well-formed x86 state, say `x86$cp`, is the conjunction of the concrete stobj recognizer `x86$cp-pre` and the well-formedness of the memory model `good-memp`. Any function, say `g`, that takes the concrete x86 state as input would need to have `x86$cp` as a guard. However, `good-memp` is an expensive function because it has to check each element of all the three memory-related fields to ensure that a good relationship among them holds. Costly guard checking slows down `g` whenever it is executed on concrete data in ACL2. We use abstract stobjs [64] to overcome these issues. An abstract stobj may be viewed as an alternative representation of a *corresponding* concrete stobj. Figure 1 illustrates the idea behind abstract stobjs, which is in the spirit of bisimulation. Let `x86` be an abstract stobj, `x86$c` a corresponding concrete stobj, and a function `f` be associated with the abstract and concrete functions `f$a` and `f$c` that update the abstract and concrete stobj, respectively. Then, $\texttt{x86\$c}_1$ corresponds to $\texttt{x86}_1$ provided that:

- `f$a` maps instance $\texttt{x86}_0$ of `x86` to $\texttt{x86}_1$.
- `f$c` maps instance $\texttt{x86\$c}_0$ of `x86$c` to $\texttt{x86\$c}_1$.
- The correspondence predicate holds for $\texttt{x86\$c}_0$ and $\texttt{x86}_0$.

Admitting an abstract stobj requires explicitly defining the correspondence predicate and discharging proof obligations that establish the correspondence of the concrete and abstract state at all times. Some examples of these are: the so-called *correspondence* theorems, that establish that the initial concrete and abstract states correspond, the concrete and abstract accessor functions for the same field produce the same value when applied to corresponding states, and the concrete and abstract updater functions for the same field produce corresponding states when applied to corresponding states; and the *preservation* theorems, which state that the recognizer always holds for the abstract state — we prove that the recognizer holds for the initial state, and it is preserved by every well-guarded updater function. For details about these theorems, see the documentation in the ACL2 manual [70]. Once an abstract stobj has been admitted, any native function accessing or updating it reduces to a simple function associated with the abstract state during reasoning and to an efficient function associated with the concrete state during execution.

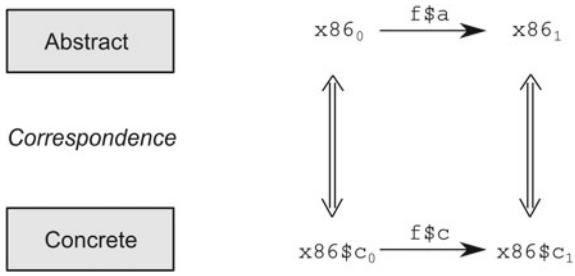


Fig. 1 Abstract and concrete x86 states

We define the x86 abstract state corresponding to the concrete state by using records [71] to model the concrete array fields. A record is a sparse association list — it is a finite normalized structure that maps keys to non-default values. Memory is represented by a single record in the abstract state; contrast this with three fields being used to represent memory in the concrete state. Functions `memi` and `!memi` invoke access and update operations (respectively) on records when reasoning and invoke `mem$ci` and `!mem$ci` (respectively) during execution. The initial representation of the abstract memory is empty, as opposed to a large linear list of zeroes for the concrete memory fields. The abstract memory contains only the values that have been written explicitly to the memory. This results in a smaller x86 state that is more amenable to reasoning, thereby solving Issue (1). The recognizer for the abstract x86 state always evaluates to true during execution, as justified by the recognizer preservation theorems. This solves Issue (2).

A benefit of using abstract stobjs is that the concrete state and its associated functions can be optimized for execution efficiency without affecting the abstract layer and other definitions of the model built on top of it, as long as the correspondence relation is maintained. Thus, maintenance of the model becomes tractable and a trade-off between reasoning and execution efficiency is avoided.

Theorems about Reads and Writes to the x86 State A program's behavior can be described by the effects it has on machine state. Given an initial state, the final state may be described as a nest of updates made in program order to the initial state. In order to reason about the behavior of a program, we need to develop lemmas to read from, write to, and re-arrange these nests of updates.

- *Read-over-Write Theorems*: There are two types of Read-Over-Write theorems. The first describes the independence or non-interference of different components of the x86 state. One example is proving that an update made to a specific register does not modify the value of any other component of the x86 state. The second asserts that reading a component after it is modified returns the value that was written to it during the modification.
- *Write-over-Write Theorems*: Like the Read-Over-Write theorems, there are two types of Write-over-Write theorems. The first asserts that independent writes to the x86 state can commute safely. The second asserts that if consecutive writes are made to a component, the most recent write is the only visible write.
- *State Preservation Theorems*: These theorems assert that writing a valid value to a component in a valid x86 state returns a modified x86 state that is still valid.

We need these theorems for every component of the x86 state, but the number of theorems required is quadratic in the number of components [72]. Adding components to the x86 state to support more features would entail proving more of these theorems. Apart from being tedious, proving such a large number of theorems can slow down the theorem prover's rewriter. Instead, we define interface functions `xr` and `xw` to read from and write to the x86 state, respectively. The functions `xr` and `xw` take field names as input; they branch on the field name and call the corresponding native accessor or updater function. Now these theorems have to be in terms of just these two functions — we have just five theorems to manage.

Though this simplifies reasoning, simulation efficiency suffers because we incur an extra function call every time the x86 state needs to be accessed or updated; every definition in the model will use these functions. Inlining `xr` and `xw` can avoid the cost of the function call, but because these functions contain a big `case` statement, this would increase the code and memory footprint of our model.

We solve this problem by using ACL2's `mbe` feature. For each component of the x86 state, we define new accessor and updater functions whose body is an `mbe`, where the `:logic` parts call `xr` and `xw` (respectively) and the `:exec` parts call the native accessor and updater functions. We then keep these new functions enabled for reasoning — when these functions are called, reasoning will be done in terms of `xr` and `xw`. We inline these functions so that execution will use the native accessors and updaters, which in turn will use the efficient concrete stobj functions. Thus, these new functions are now the top-level interfaces to the x86 state. There still remains the (small) issue of having to define two functions (a new accessor and updater) for every field added to the state. We use Lisp's (and ACL2's) ability to treat code as data and automatically generate these functions from the stobj definition.

With this mechanism in place, whenever a model developer adds a new field to the x86 state, he does not have to worry about manually proving read-over-write, write-over-write, and state-preservation theorems, and adding new accessor and updater functions that have an `mbe` in their body. This makes the model maintainable, which is critical for the longevity of a model as large as that of the x86 ISA.

3.3 Modes of Operation: Interface to the x86 *State*

As mentioned in Sect. 2.3, our x86 model offers two modes of operation: the programmer-level mode and the system-level mode. These modes can be thought of as interfaces to the x86 state; the programmer-level mode provides the same interface to the x86 state as is provided by the operating system to application programs and the system-level mode provides the same interface to programs as is provided by the processor. Thus, these modes enable simulation and reasoning about x86 machine-code programs either from the point of view of an application programmer or a kernel developer. A field in our x86 state, `programmer-level-mode`, is set to `t` when operation in the programmer-level mode is required and to `nil` otherwise.

3.3.1 Programmer-Level Mode

This mode allows the analysis of an application program while assuming that memory management, I/O operations, and other services are provided reliably by the underlying OS. We discuss some aspects of this mode below.

Memory Model In the 64-bit mode of an x86 processor, application programs use 64-bit linear addresses. The x86 ISA defines *canonical addresses* as those which

have bits 63 through 48 set to all zeros or all ones; if a non-canonical address is encountered, then a general protection exception or a stack fault occurs, depending on the context. This means that the canonical linear address ranges are 0 to $2^{47} - 1$ and $2^{64} - 2^{47}$ to $2^{64} - 1$, i.e., 2^{48} bytes of linear address space is the total available space. Note that the second canonical address range would result in addresses that are of type bignum in Lisp implementations. As discussed in Sect. 2.1.1, it is advantageous to avoid bignum creation when possible. In this case, we choose to represent linear addresses as signed integers of width 48 — these will always be fixnum values on 64-bit Lisp implementations. The linear memory range 0 to $2^{47} - 1$ is represented as is, and the other range $2^{64} - 2^{47}$ to $2^{64} - 1$ is represented by -2^{47} to -1.

In the programmer-level mode, our byte-addressable memory model discussed in Sect. 3.1 serves as the linear memory space and only up to 2^{48} bytes of the memory model can be accessed — every non-canonical linear access is an error, in accordance with the ISA. The functions `memi` and `!memi` are not used to read from or write to the memory in this mode (because they can access the entire 2^{52} bytes of available memory). Linear memory accessor and updater functions that restrict accesses to 2^{48} bytes are defined and used instead. The guards of these linear memory functions require the address to be a 48-bit signed integer, which is converted to a 48-bit unsigned integer when indexing into our memory model (for example, -1, which represents the canonical address $2^{64} - 1$, is converted to $2^{48} - 1$).

The programmer-level mode provides some support for *user-level* 64-bit segmentation, even though paging is unavailable in this mode. Segment descriptor tables (GDT and LDT) are unavailable in this mode, but if a logical address refers to the `FS` or `GS` segments (which are the only segments that can have non-zero bases in the 64-bit mode), the segment base addresses are obtained from the machine-specific registers `ia32_fs_base` or `ia32_gs_base`, respectively. It is the responsibility of system software to load these registers, and hence, they are assumed to contain appropriate values in this mode, thereby providing an environment similar to that provided by an OS to application programs.

System Calls and Non-determinism Application programs rely on various services provided by the OS. These services are usually provided by system calls. Some of these system call services are non-deterministic from the point of view of a programmer — different runs can yield different results on the same machine. For example, the `open` system call might open a file in one execution, but result in an error in another execution if the file has been deleted. The `syscall` instruction is used by application programs to call system procedures at a higher privilege level in order to request services from the underlying OS; the value in register `rax` determines which system call is to be executed. The `sysret` instruction is used by system programs to return control to the application-level program that requested the service. System call implementations differ in different OSes; a small example is that the `read` system call has the number 0 on Linux and 3 on Darwin.

The x86 instructions `syscall` and `sysret` are special in this mode. The semantic function of `syscall` is extended to provide the semantics of system calls like `read` and `write`, and the instruction `sysret` is unavailable for use. A field in our

x86 state, `os-info`, also needs to be initialized appropriately (either `:linux` or `:darwin`, which are the two OSes we currently support).

For system call simulations in the programmer-level mode, we use the return values of functions that invoke the system calls by interacting directly with the underlying OS. That is, simulation of all instructions except `syscall` occurs within ACL2; for `syscall`, we determine which system call is requested and then escape out of ACL2 to get the real results by executing that system call on the underlying system. Obviously, these functions are *impure*: logically, they are not functions because they can return different values for the same input arguments.

For reasoning about system calls, we incorporate an environment field `env` into the x86 state to represent the part of the external world that affects or is affected by system calls. The `env` field contains a specification of a subset of the *file system* as well as an *oracle* field. The oracle specifies the result of any action that is nondeterministic from the point of view of the programmer; it provides information that, though a part of the real environment, cannot be inferred from our model of the file system. An example of such information is the file descriptor of a file to be opened — an OS assigns the file descriptor depending on the number of files already opened for a particular process at the time the `open` system call is made. The oracle contains a map that associates linear addresses to a list of arbitrary values; whenever a value is needed from the oracle, the current instruction pointer is located in the map, and the first value in the corresponding list is popped off for use and removed from the list. If the program control goes to the same instruction later and the oracle is consulted, again the first value (previously the second value) in the list is popped off and then removed. Thus, to reason about a system call's effects, it is the responsibility of the user to initialize the `env` field appropriately so that it can be consulted when needed.

For example, suppose that a program makes an `open` system call to open a file, then a `write` system call to write to that file, and finally a `close` system call to close it. The `env` field's file system fields would need to be initialized to contain the file information — its name and descriptor — and the oracle field would need to associate the linear address of the first `syscall` instruction (i.e., `open`) with the value of a descriptor. Broadly speaking, reasoning about this program will involve reasoning about the following behavior: the first `syscall` instruction will pop off the descriptor from the oracle field, associate it with a file name, and change its mode as requested; the second `syscall` will use the descriptor to locate the file and write the requested number of bytes from a specified buffer to the file; and, the third `syscall` will remove the association between the file descriptor and its name. Our specifications of these system calls include accounting for error conditions too.

The contents of `env` can be symbolic. For example, to verify a program `wc` that counts the number of words in a given file, the contents of the file can be specified as an arbitrary string in the `env` field's file system fields. Of course, it is also possible to reason about concrete elements in the environment, e.g., `wc` on a concrete file, by simply initializing the `env` field with these elements. The `env` field is invisible and inaccessible when the model is used as a simulator. System call simulation and reasoning use different mechanisms and do not interfere with each other. However, the following meta-connection exists between them in our

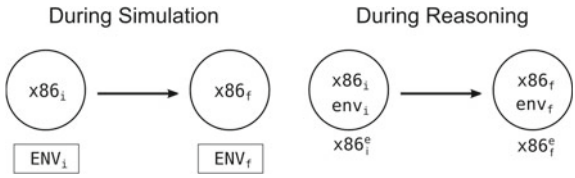


Fig. 2 Environment during simulation and reasoning

x86 model; see Fig. 2. Let $\texttt{x86}_\texttt{i}$ be an x86 state. Suppose, during simulations, the evaluation of `run(x86`$_\texttt{i}$`)` returns $\texttt{x86}_\texttt{f}$ and updates the real environment from $\texttt{ENV}_\texttt{i}$ to $\texttt{ENV}_\texttt{f}$. Then, the following is true during reasoning: if $\texttt{env}_\texttt{i}$ corresponds to $\texttt{ENV}_\texttt{i}$, and $\texttt{x86}^\texttt{e}_\texttt{i}$ refers to $\texttt{x86}_\texttt{i}$ augmented with $\texttt{env}_\texttt{i}$, then the evaluation of `run(`$\texttt{x86}^\texttt{e}_\texttt{i}$`)` during reasoning produces $\texttt{x86}^\texttt{e}_\texttt{f}$, which is $\texttt{x86}_\texttt{f}$ augmented with $\texttt{env}_\texttt{f}$, for some $\texttt{env}_\texttt{f}$ corresponding to $\texttt{ENV}_\texttt{f}$. To ensure that this connection holds for our model in the programmer-level mode, we perform co-simulations by comparing concrete program simulations to corresponding evaluations done inside ACL2, that is, using pure logical functions. More details can be found in our FMCAD'14 paper [73].

We use `env` to characterize all non-deterministic behavior, not just system calls in programmer-level mode. For example, the `RDRAND` instruction (read random number) is available both in the programmer-level and system-level modes, and for reasoning, it obtains a random value from the oracle in `env`.

3.3.2 System-Level Mode

The system-level mode is intended to be used to simulate and verify software that has supervisor privileges and access to system state, though it can also be used to verify application programs. We discuss some components of this mode below.

Physical Memory Our model includes 2^{52} bytes of byte-addressable memory; this is the largest physical address space provided by modern x86 implementations. The memory read and write functions `memi` and `!memi` pertain to physical memory.

Paging Paging is used for address translation and access rights management. In 64-bit mode, physical memory cannot be directly accessed by either system or application software. In fact, switching to the 64-bit mode requires paging to be turned on first. The system-level mode supports IA-32e paging for all configurations (4KB, 2MB, and 1GB pages) of paging data-structures, including predicates that check whether paging structure entries are valid. A linear memory address, coupled with information necessary to check access rights like the origin[4] of the address, is used as input to the paging mechanism. A linear memory access will report a failure in our model if a page-fault exception is encountered — exceptions are currently unsupported by our model. We have specified a "page walk" by defining it in terms of traversals of

[4]Determining the origin of a linear memory address constitutes answering questions like: is the linear memory reference on behalf of a read, a write, or an instruction fetch and decode? What is the privilege level, as determined by the segmentation data-structures, of this linear memory reference?.

each of the hierarchical paging structures. These traversal functions also perform on-the-fly updates to the accessed and dirty flags, as specified by the x86 ISA. Thus, in this mode, every read or write to a linear memory address goes through paging: permissions are checked to see if the memory access is allowed and the linear address is translated to its corresponding physical address.

Figure 3 represents the paging data-structures with a 4 K page configuration — we assume that the address translation does not result in a page-fault exception. The paging specification functions clearly require many slicing operations, i.e., getting and setting some ranges of bits in a number. These functions have been optimized for execution efficiency by using type declarations and guards. We use pre-existing ACL2 libraries [74] to aid in this task.

Segmentation In IA-32e mode, much of the functionality of segmentation is disabled. Segment limit checks and null segment selector checks are not performed, but descriptor table limit checks, type checks, and privilege-level checks are still carried

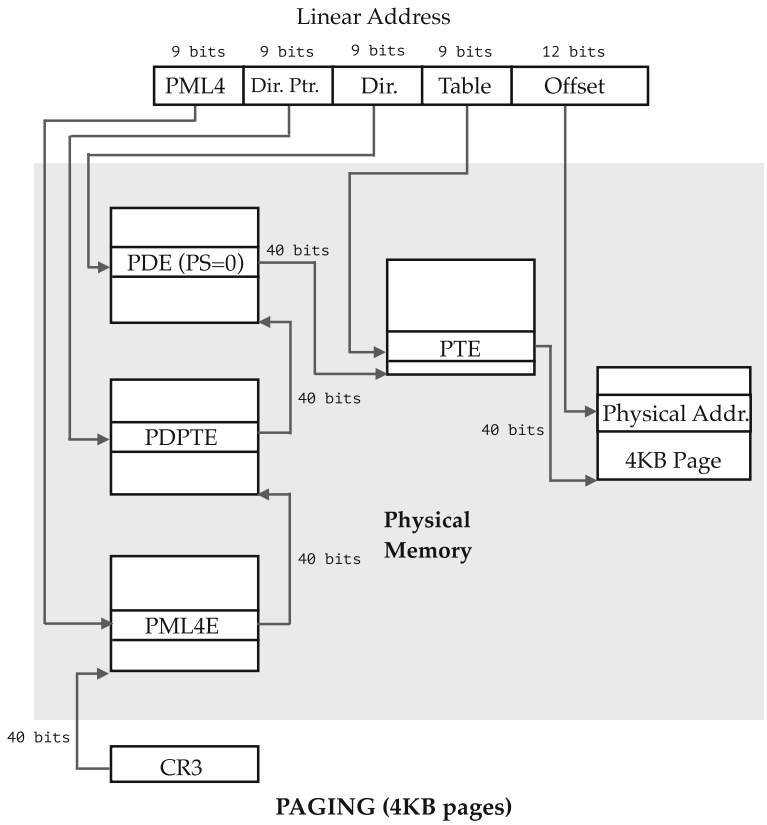


Fig. 3 IA-32e paging with 4KB page configuration

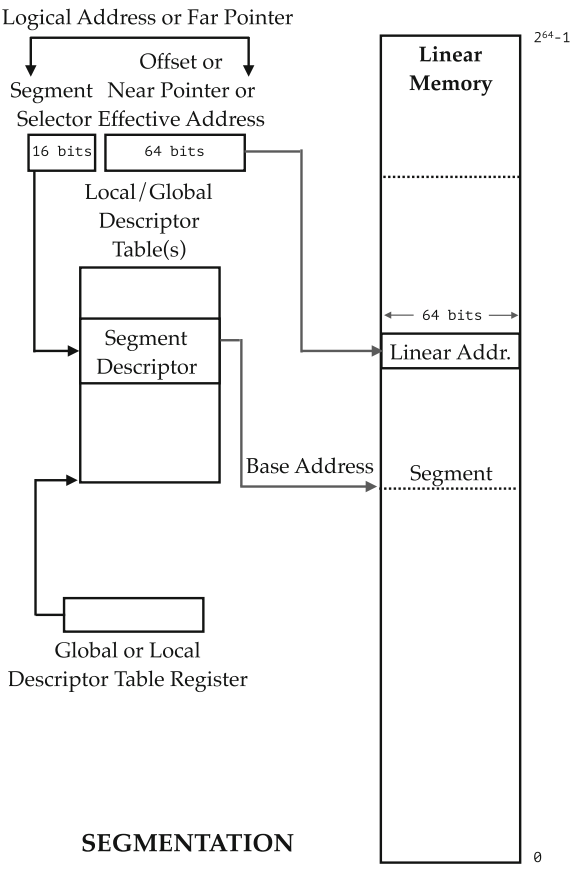


Fig. 4 IA-32e segmentation

out. The segment base for all segments except FS and GS is treated as zero; for FS and GS, logical address to linear address translation is depicted in Fig. 4.

Segment descriptors are data structures inside the descriptor tables that contain information about a segment — its size, location, access control and status information. We have defined predicates that recognize valid segment descriptors. We have also specified 64-bit segmentation-related privileged instructions like `LGDT` (load global descriptor table register) and `LLDT` (load local descriptor table register); these instructions check the descriptor table limits and the validity of the segment selector and descriptor, and then load the `gdtr` or `ldtr` register appropriately.

Segmentation plays an important role in fast system calls, i.e., `syscall` and `sysret` instructions, because these instructions involve changes in privilege levels. Unlike in the programmer-level mode, the `syscall` instruction's specification in this mode is exactly what is specified by the x86 ISA. The `sysret` instruction is also available for use in this mode. See Fig. 5 for an illustration. These instructions

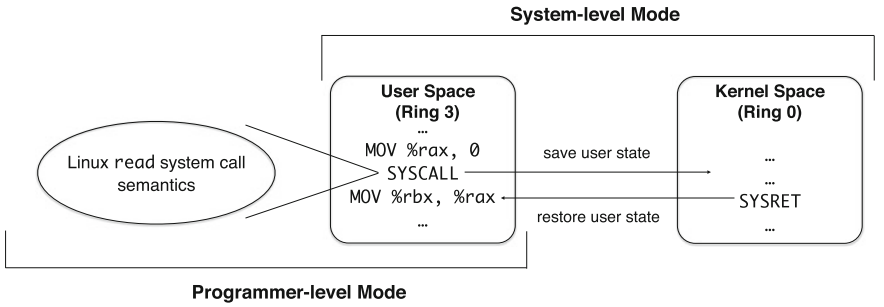


Fig. 5 Linux `read` system call in the two modes of operation of the x86 model

access system state like the machine-specific registers and code segment descriptors that describe where control is to be transferred if all privilege checks succeed.

3.3.3 Implications for Reasoning

Of all the differences from the programmer-level to system-level mode, the memory model adds the most complexity. The pseudocode for the top-level function for accessing a byte, `rm08` (which stands for "read memory — 8 bits"), is as follows. Note that `LaToPa` is a large, complex function does address translation via paging.

```
rm08(linAddr, permissions, x86):
if non-canonical-address(linAddr) then
    raiseError(x86)
else
    if programmer-level-mode(x86) then
            memi(linAddr, x86)
    else
            cpl := get-rpl-from-cs-segment(x86)
            [err, phyAddr, x86]:= LaToPa(linAddr, permissions, cpl, x86)
            if err then
               raiseError(x86)
            else
               byte := memi(phyAddr, x86)
               return (byte, x86)
            endif
    endif
endif
```

For convenience, we have similar top-level functions for accessing words, double-words, quadwords, and octawords. This means that for reasoning, we would need the usual read-over-write and write-over-write lemmas for each of these top-level functions in both the modes. Similar to our discussion about `xr` and `xw` in Sect. 3.2, we avoid the need for proving this large number of theorems using ACL2's `mbe` feature; logically, all these functions are equal to a call of another function called `rb` (read bytes). The function `rb` takes a list of linear addresses, permissions, and the

x86 state as input and if no error is encountered, returns a list of bytes at the input addresses and the x86 state. For the system-level mode, this equality to `rb` makes it possible to reason at the level of linear address space, which is akin to system programming in 64-bit mode. However, this equality is true only when the paging data-structures are set up correctly — e.g., no page fault error is encountered during `LaToPa`. This is essentially the same as stating that the programmer-level mode is an abstraction of the system-level mode as far as the memory is concerned, if the system data structures have been set up correctly.

We have adopted a similar strategy for reasoning about memory writes, where every write is logically equal to a call of `wb` (write bytes). Another benefit of using `rb` and `wb` is that all memory accesses are reduced to reasoning about list operations (like subset) as opposed to tedious arithmetic involving address ranges. Using lists in `rb` and `wb` does not affect the simulation efficiency of the model because these functions are used only during reasoning; for simulation, efficient stobj code is used.

3.4 Instruction Semantic Functions

Functions like `rm08` (see Sect. 3.3.3 above) abstract away the details of accessing the x86 state, irrespective of the mode of operation. This makes specifying the behavior of instructions relatively straightforward.

The instruction fetch, decode, and execute function (described in Sect. 3.5) passes on the decoded parts of the instruction to the appropriate instruction semantic functions, which use them to determine the locations of the operands (if any) and then fetches them from the x86 state. These functions then call an *operation specification function* — e.g., the `ADD` operation's specification function takes two operands and the flags as input and returns the result and the output flags. The instruction semantic function then writes the outputs from this specification function to the appropriate parts in the x86 state. A benefit of this approach is that instructions which fetch inputs and write outputs in the same manner can share the same semantic function; the actual computation is done by calling the correct operation specification function inside the semantic function. For example, the following instructions (and opcodes) have a single semantic function but different operation specification functions: `ADD` (`0x00`, `0x01`), `OR` (`0x08`, `0x09`), `ADC` (`0x10`, `0x11`), `SBB` (`0x18`, `0x19`), `AND` (`0x20`, `0x21`), `SUB` (`0x28`, `0x29`), `XOR` (`0x30`, `0x31`), `CMP` (`0x38`, `0x39`), and `TEST` (`0x84`, `0x85`). Again, this facilitates maintainability.

Characterizing Undefined Behavior Some x86 instructions leave certain parts of the machine state undefined. For example, according to the Intel manuals, the carry, parity, auxiliary, zero, sign, and overflow flags are all undefined after an unsigned divide instruction (`DIV`) is executed. A property of an "undefined value" is that an equality test with another value, either defined or undefined, should be indeterminate. This also implies that every undefined value should be *unique*, in the sense that any two such values are independent of each other. An illustrative example of why

indeterminateness and *uniqueness* of undefined values is important is as follows: if a program contains a DIV instruction followed by a JGE instruction (which transfers control to a given address if the sign flag and overflow flag are equal, otherwise the control goes to the instruction following JGE), our analysis should not be able to determine whether the jump occurred or not. This alerts us to unreliable, if not dangerous, behavior that a program may exhibit. Thus, the specification of instructions like DIV should include such undefined behavior.

One way to model undefined behavior is to use 4-valued logic, where the unknown X meets our criteria. However, an issue with using 4-valued logic in our model is that we would need to use at least two bits to represent every bit in the processor state, thereby doubling the memory footprint of our model. Also, 4-valued logic is more complicated to reason with because X has a tainting effect. The usual 2-valued logic equal function cannot be used in cases where either or both of its input values can be X, in which case the result should also be X.

Instead, we model undefined behavior by defining a constrained function (see Sect. 2.1.1) called create-undef to provide indeterminateness and by adding a field called undef to our x86 state to provide uniqueness. Create-undef has an input and output arity of one; the only thing we know about the output is that it is an unsigned integer, irrespective of the input. Note that if the input to two calls of create-undef are different, then we can not determine if they are equal. The undef field contains an unsigned integer that is incremented every time an undefined value is required; thus, we get a unique value. We put undef and create-undef together by wrapping them up in another function called undef-read, which takes an x86 state as input, and returns a unique undefined value and a modified x86 state where undef is one more than what it was in the input x86 state. We ensure that undef-read is the only function that can modify the undef field by making the undef updater function, update-undef, untouchable (see Sect. 2.1.1 for details). Using undef-read will force us to reason about all the possible behaviors if a program relies on undefined values.

```
undef-read(x86):
        undef-seed         := undef(x86)
        new-undefined-val  := create-undef(undef-seed)
        new-x86            := update-undef(1 + undef-seed)
        return(new-undefined-val, new-x86)
```

Characterizing undefined behavior is a bit different for concrete program simulations. Revisiting our DIV and JGE example, if that program runs on a real x86 machine, it would do exactly one of the following: jump to the given address or give control to the instruction following JGE. A simulator should behave in the same way as a real x86 processor; we do not have the luxury of accounting for all possible behaviors here. One way to solve this problem is to perform *native execution*. If our model is being used as a simulator on an x86 machine, then for every instruction that leaves any component undefined, we can escape from ACL2, re-run the same instruction on the real x86 machine, gather the values of the components that are supposed to be undefined, and pass those values up to the x86 model. These values

can then be plugged into the corresponding fields in the x86 state in our model. This would require a trust tag (see Sect. 2.1.1) when the model is in use as a simulator. However, switching environments can significantly slow down the simulation speed of our x86 model. Instead, when our model is used as a simulator, we chose simply to assign suitable concrete values to the x86 components that are supposed to be undefined by an instruction. We also log the value of the instruction pointer when this occurs so that we have a record of which instructions were simulated that concretized any undefined behavior in this manner. The concrete value is chosen based on tests performed on a real x86 machine that indicate the most probable value for that instruction. This optimization is an example of balancing efficiency and accuracy; the simulator is accurate in that it captures a behavior that is indeed possible and it is efficient because the entire simulation takes place in ACL2/Lisp.

Note that the principle behind the oracle in the `env` field (see Sect. 3.3.1) is quite similar to that of the `undef` field. We can imagine discarding the `undef` field completely and using the oracle field in `env` to provide undefined values. However, the `undef` and `env` fields are used in different situations. For reasoning about programs that involve commonly occurring undefined events (like flag computations), using the oracle field can be tedious because it has to be initialized appropriately, i.e., a list of values has to be associated with an address which contains an instruction that involves any undefined behavior. Imagine doing that for all the undefined flags whenever `DIV` is executed. The `undef` field does not need to be initialized. The initialization of the oracle field provides a way of tracking any computation that relies on the external environment. Such computations do not happen often, and when they do, this initialization specifies exactly what we expect from the environment. In the case of undefined values, we do not have any such expectations — all we want is an infinite pool of unique and indeterminate values.

3.5 *Step and Run Functions*

The step function of our x86 ISA model fetches, decodes, and executes a machine instruction. It takes an x86 state as input, fetches the instruction from the memory at the location pointed to by the instruction pointer `rip`, and then dispatches control to the appropriate instruction semantic function. The x86 ISA has variable-length instructions, as depicted in Fig. 6 — the maximum length of a legal x86 instruction is

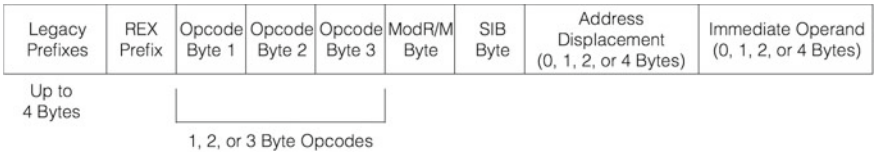


Fig. 6 x86 instruction format (64-bit mode)

15 bytes. Since most instructions are smaller than this limit, our step function lazily fetches one byte of an instruction at a time instead of fetching 15 bytes at once. Just enough bytes are fetched and decoded to determine the instruction opcode (so that the right instruction semantic function can be called), and the sizes and location of the operands. Decoding a byte of the instruction gives information about the nature of the next byte. E.g., if the first byte is a one-byte opcode, then the ISA tells us if that opcode requires a ModR/M byte. If it does not, then we dispatch control to the proper instruction semantic function, which, if needed, fetches the address displacement or immediate data encoded in the instruction or the operands from the x86 state. If the one-byte opcode requires a ModR/M byte, then the byte following the opcode is the ModR/M byte, which gives us yet more information about whether a SIB byte is expected, and so on.

The run function is the top-level specification function of our x86 ISA model.

```
x86-run(n, x86):
        if (ms x86) || (n == 0) then
           // Problem indicated by the model state field or
           // no more instructions left to be run
           x86
        else
           x86-run(n - 1, x86-fetch-decode-execute(x86))
        endif
```

4 Model Validation

A high simulation speed facilitates efficient co-simulations, which increase trust in the accuracy of the model, as discussed in Sect. 2.3. An added benefit of having two modes of operation of the x86 model is an increased simulation speed in the programmer-level mode, where much of the x86 state is abstracted away. For example, in the system-level mode, every access to the memory requires yet more accesses to the hierarchical data structures in the physical memory for memory management, whereas a memory access in the programmer-level mode is a direct operation.

The simulation speed of our model is ∼3.3 million instructions/second in the programmer-level mode and ∼330,000 instructions/second in the system-level mode.[5] We have successfully simulated a SAT solver,[6] which produced exactly the same effects as the real machine on the model's memory and registers.

Figure 7 illustrates our framework for performing co-simulations for model validation. Given a machine-code program in an executable format, we first determine if it can be simulated on our model. We then parse the executable file using our ACL2-based parser that determines, among other things, where the program is to be placed into the memory of our x86 state. Our ACL2-based loader loads the program

[5] This was measured on a machine with Intel Xeon E31280 CPU @ 3.50 GHz with 32GB RAM.

[6] This solver, developed by Marijn Heule, has performance comparable to state-of-the-art solvers.

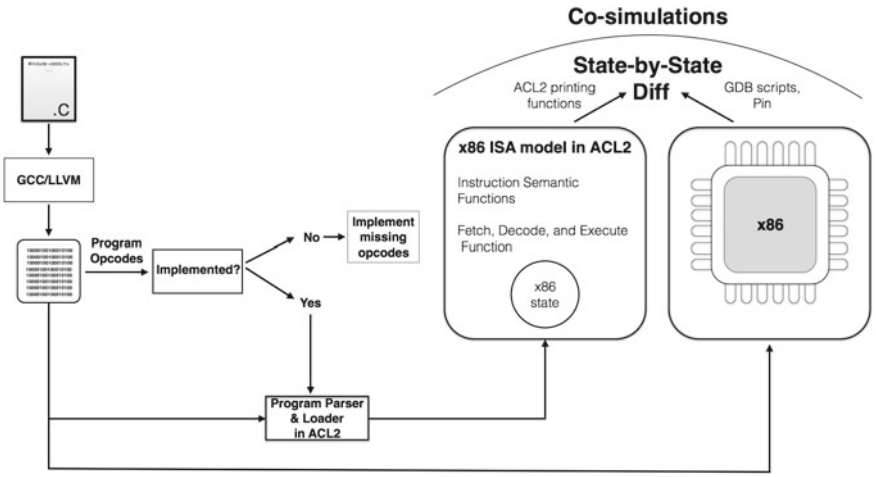


Fig. 7 Model validation via co-simulations

into the x86 state appropriately. The parser and loader are discussed in Sect. 4.1. We then execute the program on our model as well as on the real machine, taking care to initialize the state in our model appropriately (see Sect. 4.1.1 for details). The states of both the model and the machine are then logged in the same manner, i.e., either after every instruction or at any other granularity using instrumentation tools on the real machine and their equivalent on our model (Sect. 4.2). Any differences found between the model and the real machine are logged for later examination.

4.1 Machine Program Parser and Loader

We chose to write our machine program parser and loader in ACL2 itself. Writing our tools in ACL2 leaves open the possibility of formally reasoning about them. Also, we find it more convenient to debug and maintain ACL2 code — the books containing these tools are regularly [75] certified by ACL2.

Our parser and loader support machine-code programs in either ELF [76] format for Unix systems or Mach-O [77] format for Darwin systems. The parser reads the executable file, checks whether it is well-formed, and uses its headers to retrieve the various sections (e.g., text, data) in the file, along with information like their location in the memory. The loader takes this output from the parser and an initial x86 state as input, and places the sections in the right locations in the model's memory.

4.1.1 x86 State Initialization

For co-simulations, the initial state of the model should match the initial state of the real machine. The parser and loader only initialize the memory in the x86 state of our model. For convenience, we provide a function called `init-x86-state` to initialize all the components of the x86 state at once. This function also takes in an address, which, if encountered as an instruction pointer, halts the simulation of the program. Thus, simulating only a part of the program is also possible.

If simulation is being done in the system-level mode of the model, we provide a default paging data-structure configuration (identity-mapped 1GB pages). Of course, users are free to load their own system data structure configurations as well.

4.2 Instrumentation

For more control over the granularity at which the states of the model and a real machine are compared during co-simulations, the program must be instrumented in the same manner on the model and the machine. We use tools like the GNU Debugger and Intel's Pin [65] to instrument programs on a real machine. We have developed our own ACL2-based program instrumentation tools to provide similar capabilities for our x86 model. Our instrumentation tools are very flexible; e.g., we can trace every read and/or write to the x86 state (including the memory) or only trace those reads and writes that meet certain criteria. We can step through a program instruction by instruction, similar to the `nexti` and `stepi` commands on GDB, we can execute a specified number of instructions, and we can set breakpoints at arbitrary points in the program (e.g., when `rax` = 4 or when `rip` = 0x400952, etc.).

Apart from aiding in co-simulations, our instrumentation tools help in program comprehension. Our model is a programmable simulator — it provides a safe environment to simulate the program and examine its effects on the machine state.

5 Reasoning About x86 Machine-Code Programs

This chapter is primarily about the development and validation of our x86 ISA model, but because optimizing reasoning efficiency affected our modeling decisions, in this section we briefly discuss x86 machine-code verification using our model.

We have developed general libraries to reason about x86 machine code, both in the programmer-level mode and the system-level mode. An example of such a library is one which aids in reasoning about the non-interference/overlap of memory regions. The functions `rb` and `wb` (see Sect. 3.3.3) operate in terms of lists of addresses and bytes, making non-interference and overlap expressible in terms of notions like disjoint and subset. These libraries are mature enough to reason about non-trivial properties of a given program automatically, e.g., independence of the stack and

heap from the program and data. We also have a library to reason about traversals of and updates to paging data-structures. We have formulated predicates that recognize a valid paging structure entry, and proved that if these predicates hold on a given entry, walking that entry will not result in a page-fault exception. Regardless of on-the-fly updates to these data structures during traversals (accessed and dirty bits), we have proved that repeated walks of valid entries do not modify the address mapping. Such lemmas enable reasoning at the level of linear memory reads and writes in the system-level mode. This is important because in IA-32e mode, even system software cannot access physical memory directly — linear memory is the abstraction that must be used.

We can automatically verify snippets of straight-line machine code using a BDD or SAT-based bit-blasting proof engine in ACL2 called GL [16, 78]. Providing the specification is the primary user requirement in order to prove theorems using this framework. If a conjecture fails, GL can compute counterexamples. Using this technique, we have verified a complicated population count (bit-count) program [79], and we detected a bug in an incorrect version [80]. Such an automated symbolic execution technique for reasoning about straight-line code can facilitate compositional verification, thereby reducing the cost of reasoning about larger programs.

A verification effort of note in the programmer-level mode is that of a word-count program [73], which computes the number of characters, words, and lines in a stream read in from the user using the `read` system call. We wrote a trio of simple ACL2 specification functions that compute these counts of a string. The final theorem asserts that the values returned by these three specification functions on standard input are found in the expected memory locations of the final x86 state, which is obtained by symbolically running the word-count program on our x86 model. Using our lemma libraries, we automated the proof of disjointness of the word-count program and its stack in every execution. We proved that irrespective of the size of the input file, this program always uses a fixed amount of memory on the stack to compute and store the counts. Another proof that was discharged automatically was that the word-count program does not modify unintended regions of memory, i.e., the only writes that occur during the program's execution are to the stack and the rest of the memory is the same as it was before the execution. Our lemma libraries reduced the manual effort required for machine-code verification of this program substantially, as demonstrated by some empirical evidence. The lines of ACL2 needed to verify the program without our libraries were appropriately 20 K, but with the libraries were appropriately 8 K. Even though 8 K lines may still seem inordinate, it should be noted that more than half of these 8 K lines were generated by ACL2 in response to requests to simplify certain symbolic expressions related to the execution of multiple instructions. These simplified expressions are large because a typical instruction makes many updates to the x86 state. The effort with 20 K lines also had around 4 K lines resulting from simplification of symbolic expressions.

In the future, we hope to use recently developed ACL2 tools like Codewalker [58] and Stateman [81] to increase automation. Codewalker is a machine-code decompilation tool — it automatically lifts machine code to a higher level of abstraction (i.e., ACL2 functions), given the operational semantics of the ISA. Stateman manages

large terms that represent machine states — it reduces the overhead of simplifying these terms by projecting out only the relevant parts.

6 Related Work

Our main thrust for developing an x86 ISA formal model is to enable program verification. Though program verification has a long history, with Turing's 1949 paper [82] being one of the earliest works, we briefly discuss only those efforts that involved the development and validation of formal simulators for processors.

Our modeling and reasoning strategy has been heavily influenced by the CLI stack [14], even though it was composed of systems that are simpler than modern ones. The CLI stack included NQTHM-based formal simulators of both the gate-level design and the ISA of a microprocessor called FM9001 [83], an assembler with linking loader for a language called Piton [84] that targets this microprocessor, and a higher-level language, micro-Gypsy [85], that targets Piton. Each of these simulators was validated and used as a reasoning framework to prove the correctness of the system "above" it. Another project that influenced our work was the formalization [86] of most of the user-mode instruction set of a commercial Motorola MC68020 microprocessor by Boyer and Yu in NQTHM; this formal model was used to verify machine code produced by compiling the Berkeley string library using GCC. Our work on formalizing the Y86 [87], a simple 32-bit x86-like processor that was developed by Bryant and O'Hallaron for pedagogical purposes, can be called the precursor of our x86 ISA model; we used our ACL2-based Y86 model as a prototype to evaluate the design decisions we made for our x86 ISA model. This Y86 model is also released [88] as a part of the ACL2 Community Books.

Many simulators have been built for semi-formal and formal purposes. Formal models of processor ISAs have been used as a target specification for microprocessor design verification [9, 10, 27, 83]. Rockwell Collins built a symbolic simulator [89] for their JEM1 microprocessor in the PVS [90] theorem proving system to detect microcode errors. Rockwell also published studies [91, 92] from an engineering standpoint of how to efficiently simulate formal models of processors in ACL2, again using the JEM1 microprocessor as an example. Formal models of mainstream commercial multiprocessors like ARM [93], PowerPC [94], and x86 [95–97] have been built in the HOL [98] theorem prover to develop rigorous semantics of relaxed-memory concurrency that are provided by these modern architectures. HOL-based ISA-level specifications of these mainstream processors (e.g., ARMv7 [99]) have been used to verify machine-code programs automatically using Myreen's "decompilation into logic" technique [56, 59, 100], which reduces the problem of reasoning about machine code to reasoning about simpler logic functions. The verification effort for seL4 [101, 102], the "world's first operating-system kernel with an end-to-end proof of implementation correctness and security enforcement", was initially done on formal models of ARMv6 and x86 using Isabelle/HOL. Morrisett et al. developed a framework for software fault isolation [55] that involved building a Coq-based x86

ISA specification that can be used for machine-code verification. Morrisett has also been focused on building scalable formal models for reasoning [103]. The Coq proof assistant [104] was used by Feng et al. to build a simplified formal model of the x86 processor in order to verify machine code using domain-specific and separation logics [105]. Shao's recent efforts to develop and certify clean-slate OS kernels [106] has also involved modeling processor architectures in Coq. There have been investigations into developing domain-specific languages [107, 108] to facilitate clear and precise specification of ISAs, even by non-experts, to reduce the possibility of introducing modeling errors.

Simulators used for testing have also received attention — so much so that formal methods have been employed to ensure that they are faithful to the hardware. A Coq formalization [109] of the ARM instruction set and addressing modes was done as a part of a project to certify a System-on-Chip simulator `SimSoC`. A C model of `SimSoC` was obtained in Coq using the C semantics provided by the CompCert project [110]. Parts of these two models were then proved equivalent in Coq.

7 Conclusion

We have presented our executable model of the x86 ISA, which formalizes an interpreter-style operational semantics in ACL2. The contribution of our work is a unified x86 ISA model for both simulation and formal analysis of x86 machine-code programs. It can also be used as a target specification to verify whether a micro-architecture implements this ISA. Our unified model offers two modes of operation, which allow choosing the level of rigor with which analysis will be performed. Formal models written in most higher-order theorem proving systems require the extraction of executable specifications for validation via co-simulations; thus, trusting or verifying the extraction process is necessary. Our ACL2-based formal specifications of the x86 ISA are directly executable; we also provide tools that aid in dynamic program instrumentation. Though higher-order systems offer more expressibility, applications like specifying processor ISAs are often more "natural" to do in first-order logic. Models written in first-order logic look very like the procedural code developers without experience in formal methods work with, which facilitates the adoption of tools based on these models.

Properties about our model are certified using ACL2. We have proved many properties (e.g., full functional correctness, memory usage analysis) of several application-level programs. We are currently working on making our framework more amenable to reasoning about programs with supervisor privileges — such programs directly impact system security. At the same time, we continue to extend our model by supporting more features of the x86 ISA. The latest version of our framework is available online [12].

Our work is in the spirit of the CLI and ProCoS [111, 112] projects. In this chapter, instead of verification techniques, we have concentrated on the engineering aspect of formal tool construction. Though developing and improving verification techniques

is critical to achieve software reliability, it is equally important to examine our tools. Modern systems are often large and complex, as a result of which there is a considerable amount of engineering involved before the groundwork for useful research can be laid. We hope that documenting the insights we gained while building our formal framework will encourage others to discuss and share their own engineering efforts.

Acknowledgements This project was supported by the DARPA CRASH program under N66001-10-2-4087, and preparation of this chapter was also partially supported by the DARPA MUSE program under FA8750-15-C-0007. We are currently supported by NSF SaTC program under CNS-1525472. We thank Marijn Heule for his feedback on the chapter.

References

1. von Neumann, J.: First draft of a report on the EDVAC. IEEE Ann. Hist. Comput. **15**(4), 27–75 (1993). http://doi.ieeecomputersociety.org/10.1109/85.238389
2. Turing, A.M.: On computable numbers, with an application to the Entscheidungsproblem. J. Math. **58**(345–363), 5 (1936)
3. Rojas, R.: Konrad Zuse's legacy: the architecture of the Z1 and Z3. Ann. Hist. Comput. IEEE **19**(2), 5–16 (1997)
4. Intel Manuals (September, 2015) Intel 64 and IA-32 Architectures Software Developer's Manuals. Order Number: 325462-056US
5. Intel (Accessed: October, 2015.) Intel Developer Zone - ISA Extensions. See https://software.intel.com/en-us/isa-extensions/
6. Kaufmann, M., Moore, J.S.: (Accessed: 2015) ACL2 home page. (see http://www.cs.utexas.edu/users/moore/acl2)
7. Kaufmann, M., Manolios, P., Moore, J.S.: Computer-Aided Reasoning: An Approach. Kluwer Academic Publishers, Boston (2000)
8. Boyer, R.S., Kaufmann, M., Moore, J.S.: The boyer-moore theorem prover and its interactive enhancement. Comput. Math. Appl. **29**(2), 27–62 (1995). http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.51.7689
9. Sawada, J., Hunt Jr., J.W.: Verification of FM9801: an out-of-order microprocessor model with speculative execution, exceptions, and program-modifying capability. Form. Methods Syst. Des. **20**(2), 187–222 (2002). http://dl.acm.org/citation.cfm?id=584665
10. Hunt Jr., W.A.: FM8501: A Verified Microprocessor, LNAI, vol. 795. Springer (1994)
11. Section 2.2.10: Intel 64 Architecture, Vol. 1, Intel 64 and IA-32 Architectures Software Developer's Manual. (September, 2015) Order Number: 325462-056US
12. x86isa Books in the ACL2 Community Books Project on Github (Accessed: October, 2015). See https://github.com/acl2/acl2/tree/master/books/projects/x86isa
13. x86isa ACL2 Books (Accessed: October, 2015) Documentation of the bleeding-edge ACL2-based model of the x86 ISA. http://www.cs.utexas.edu/users/moore/acl2/manuals/current/manual/index.html?topic=ACL2____X86ISA
14. Bevier, W.R., Hunt Jr., W.A., Moore, J.S., Young, W.D.: Special issue on system verification. J. Autom. Reason. **5**(4), 409–530 (1989)
15. Kaufmann, M., Moore, J.S., Ray, S., Reeber, E.: Integrating external deduction tools with ACL2. J. Appl. Log. **7**(1), 3–25 (2009). doi:10.1016/j.jal.2007.07.002, http://www.sciencedirect.com/science/article/pii/S1570868307000602, special Issue: Empirically Successful Computerized Reasoning
16. Swords, S.: A Verified Framework for Symbolic Execution in the ACL2 Theorem Prover. PhD thesis, Department of Computer Sciences, The University of Texas at Austin (2010). http://repositories.lib.utexas.edu/handle/2152/ETD-UT-2010-12-2210

17. Glucose SAT Solver (Accessed: October, 2015). http://www.labri.fr/perso/lsimon/glucose/
18. Minisat SAT Solver (Accessed: October, 2015). http://minisat.se/
19. ACL2 System and Books Repository on Github (Accessed: October, 2015). See https://github.com/acl2/acl2
20. Davis, J., Kaufmann, M.: Industrial-Strength Documentation for ACL2. EPTCS 152:9–25 (2014). http://www.cs.utexas.edu/users/kaufmann/talks/acl2-workshop-2014/acl2-14-davis-kaufmann.pdf
21. Kaufmann, M., Moore, J.S.: (Accessed: October, 2015) ACL2 documentation. See http://www.cs.utexas.edu/users/moore/acl2/current/manual/index.html?topic=ACL2____ACL2
22. 2005 ACM Software System Award (2005) The Boyer-Moore Theorem Prover. (see http://awards.acm.org/software_system/)
23. ACL2 Applications (Accessed: October, 2015). See http://www.cs.utexas.edu/users/moore/acl2/current/manual/index.html?topic=ACL2____INTERESTING-APPLICATIONS
24. Centaur Technology (Accessed: October 2015). http://www.centtech.com
25. FV Group at Centaur (Accessed: October, 2015). http://fv.centtech.com
26. Davis, J., Slobodova, A., Swords, S.: Microcode verification — another piece of the microprocessor verification puzzle. In: Klein, G., Gamboa, R. (eds.) Interactive Theorem Proving, Lecture Notes in Computer Science, vol. 8558, pp. 1–16. Springer International Publishing (2014). doi:10.1007/978-3-319-08970-6_1
27. Hunt Jr., W.A., Swords, S., Davis, J., Slobodova, A.: Use of formal verification at centaur technology. In: Hardin, D.S. (ed.) Design and Verification of Microprocessor Systems for High-Assurance Applications, pp. 65–88. Springer (2010). https://www.cs.utexas.edu/~jared/publications/2010-hardin-centaur.pdf
28. Flatau, A., Kaufmann, M., Reed, D.F., Russinoff, D., Smith, E.W., Sumners, R.: Formal verification of microprocessors at AMD. In: 4th International Workshop on Designing Correct Circuits (DCC 2002), Grenoble, France (2002)
29. Russinoff, D.M.: A case study in formal verification of register-transfer logic with ACL2: the floating point adder of the AMD athlon TM processor. In: Formal Methods in Computer-Aided Design, pp. 22–55. Springer (2000)
30. Russinoff, David: Computation and formal verification of SRT quotient and square root digit selection tables. IEEE Trans. Comput. **62**(5), 900–913 (2013)
31. Russinoff, D., Kaufmann, M., Smith, E., Sumners, R.: Formal verification of floating-point RTL at AMD using the ACL2 theorem prover. In: Proceedings of the 17th IMACS World Congress on Scientific Computation, Applied Mathematics and Simulation, Paris, France (2005)
32. Reeber, E., Sawada, J.: Combining ACL2 and an automated verification tool to verify a multiplier. In: Proceedings of the Sixth International Workshop on the ACL2 Theorem Prover and Its Applications, pp. 63–70. ACM (2006)
33. Sawada, J., Reeber, E.: ACL2SIX: a hint used to integrate a theorem prover and an automated verification tool. In: Formal Methods in Computer Aided Design, 2006. FMCAD '06, pp. 161–170 (2006). doi:10.1109/FMCAD.2006.3
34. Sawada, J., Gamboa, R.: Mechanical verification of a square root algorithm using Taylor's theorem. In: Formal Methods in Computer-Aided Design, pp. 274–291. Springer (2002)
35. Sawada, J., Sandon, P., Paruthi, V., Baumgartner, J., Case, M., Mony, H.: Hybrid verification of a hardware modular reduction engine. In: Proceedings of the International Conference on Formal Methods in Computer-Aided Design, FMCAD Inc, Austin, TX, FMCAD '11, pp. 207–214 (2011). http://dl.acm.org.ezproxy.lib.utexas.edu/citation.cfm?id=2157654.2157686
36. O'Leary, J.W., Russinoff, D.M.: Modeling algorithms in SystemC and ACL2. In: Verbeek, F., Schmaltz, J. (eds.) Proceedings Twelfth International Workshop on the ACL2 Theorem Prover and its Applications, Vienna, Austria, 12-13th July 2014, Open Publishing Association, Electronic Proceedings in Theoretical Computer Science, vol. 152, pp. 145–162 (2014). doi:10.4204/EPTCS.152.12
37. Coglio, A.: Second-order functions and theorems in ACL2. In: Kaufmann, M., Rager, D.L. (eds.) Proceedings Thirteenth International Workshop on the ACL2 Theorem Prover and Its

Applications, Austin, Texas, USA, 1-2 October 2015, Open Publishing Association, Electronic Proceedings in Theoretical Computer Science, vol. 192, pp. 17–33 (2015). doi:10.4204/EPTCS.192.3
38. Selfridge, S., Smith, E.W.: Polymorphic types in ACL2. In: Verbeek, F., Schmaltz, J., (eds.) Proceedings Twelfth International Workshop on the ACL2 Theorem Prover and its Applications, Vienna, Austria, 12-13th July 2014, Open Publishing Association, Electronic Proceedings in Theoretical Computer Science, vol. 152, pp. 49–59 (2014). doi:10.4204/EPTCS.152.4
39. Rager, D.L., Ebergen, J., Lee, A., Nadezhin, D., Selfridge, B., Chau, C.K.: A brief introduction to oracle's use of ACL2 in verifying floating-point and integer arithmetic. In: Kaufmann, M., Rager, D.L. (eds.) Proceedings Thirteenth International Workshop on the ACL2 Theorem Prover and Its Applications, Austin, Texas, USA, 1-2 October 2015, Open Publishing Association, Electronic Proceedings in Theoretical Computer Science, vol. 192 (2015)
40. Greve, D.: Address enumeration and reasoning over linear address spaces. In: Fifth International Workshop on the ACL2 Theorem Prover and Its Applications (ACL2-2004) (2004)
41. Greve, D., Richards, R., Wilding, M.: A summary of intrinsic partitioning verification. In: 5th International Workshop on the ACL2 Theorem Prover and Its Applications (ACL2 2004), Austin, TX (2004)
42. Hardin, D.S., Smith, E.W., Young, W.D.: A robust machine code proof framework for highly secure applications. In: Proceedings of the Sixth International Workshop on the ACL2 Theorem Prover and Its Applications, pp. 11–20. ACM, New York, NY, USA, ACL2 '06 (2006). doi:10.1145/1217975.1217978
43. CLHS (Accessed: October, 2015) Common Lisp HyperSpec. http://www.lispworks.com/reference/HyperSpec/index.html
44. ACL2 Feature: Guards (Accessed: October, 2015). See http://www.cs.utexas.edu/users/moore/acl2/manuals/current/manual/?topic=ACL2____GUARD
45. Greve, D.A., Kaufmann, M., Manolios, P., Moore, J.S., Ray, S., Ruiz-Reina, J.-L., Sumners, R., Vroon, D., Wilding, M.: Efficient execution in an automated reasoning environment. J. Funct. Program. **18**(1) (2008)
46. System Class Integer (Accessed: October, 2015) CLHS, Common Lisp HyperSpec. http://www.lispworks.com/documentation/HyperSpec/Body/t_intege.htm
47. ACL2 Feature: Trust Tags (Accessed: October, 2015). See http://www.cs.utexas.edu/users/moore/acl2/manuals/current/manual/?topic=ACL2____DEFTTAG
48. ACL2 Feature: Untouchable Functions (Accessed: October, 2015). See http://www.cs.utexas.edu/users/moore/acl2/manuals/current/manual/?topic=ACL2____PUSH-UNTOUCHABLE
49. The CompCert Project (Accessed: October, 2015). The CompCert C Compiler. http://compcert.inria.fr/compcert-C.html
50. Linux Memory Management (Accessed: October, 2015). Linux System Administrators Guide. http://www.tldp.org/LDP/sag/html/memory-management.html
51. McKusick, M.K., Neville-Neil, G.V., Watson, R.N.M.: Chapter 6: Memory Management. The Design and Implementation of the FreeBSD Operating System, Addison Wesley Professional (2014)
52. Kerrisk, M.: The Linux Programming Interface: A Linux and UNIX System Programming Handbook, 1st edn. No Starch Press, San Francisco (2010)
53. Windows Memory Management (Accessed: October, 2015.) Windows System Administrators Guide. http://msdn.microsoft.com/en-us/library/windows/desktop/aa366779(v=vs.85).aspx
54. Yu, D., Shao, Z.: Verification of safety properties for concurrent assembly code. In: Proceedings of 2004 International Conference on Functional Programming (ICFP'04) (2004)
55. Morrisett, G., Tan, G., Tassarotti, J., Tristan, J.-B., Gan, E.: RockSalt: better, faster, stronger SFI for the x86. In: Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation, pp. 395–404. ACM, PLDI '12 (2012). doi:10.1145/2254064.2254111

56. Myreen, M.O.: Formal Verification of Machine-code Programs. PhD thesis, University of Cambridge, Computer Laboratory, Trinity College (2008). http://www.cl.cam.ac.uk/~mom22/thesis.pdf
57. Smith, E.W.: Axe, An Automated Formal Equivalence Checking Tool For Programs. PhD thesis, Department of Computer Science, Stanford University (2011)
58. Moore, J.S.: (Accessed: October, 2015) Codewalker. See https://github.com/acl2/acl2/books/projects/codewalker
59. Myreen, M.O., Gordon, M.J.C., Slind, K.: Decompilation into logic - improved. In: Formal Methods in Computer-Aided Design (FMCAD), 2012, pp. 78–81 (2012). http://www.cs.utexas.edu/~hunt/FMCAD/FMCAD12/016.pdf
60. Bellard, F.: QEMU, a fast and portable dynamic translator. In: USENIX Annual Technical Conference, FREENIX Track, pp. 41–46 (2005)
61. Lawton, K.P.: Bochs: A Portable PC Emulator for Unix/X. Linux J 1996(29es) (1996). http://dl.acm.org/citation.cfm?id=326350.326357
62. Unicorn (Accessed: October, 2015) Unicorn: Slides from BlackHat USA 2015. http://www.unicorn-engine.org/BHUSA2015-unicorn.pdf
63. AMD Manuals (Accessed: October, 2015.) AMD64 Architecture: Developer Guides, Manuals and ISA Documents. http://developer.amd.com/resources/documentation-articles/developer-guides-manuals/
64. Goel, S., Hunt Jr., W.A., Kaufmann, M.: Abstract stobjs and their application to ISA modeling. In: Proceedings of the ACL2 Workshop 2013, EPTCS 114, pp. 54–69 (2013). http://arxiv.org/pdf/1304.7858.pdf
65. Luk, C.-K., Cohn, R., Muth, R., Patil, H., Klauser, A., Lowney, G., Wallace, S., Reddi, V.J., Hazelwood, K.: Pin: Building Customized Program Analysis Tools with Dynamic Instrumentation. SIGPLAN Not **40**(6), 190–200 (2005). doi:10.1145/1064978.1065034
66. Kaufmann, M., Hunt Jr., W.A.: Towards a Formal Model of the x86 ISA. Technical report, Department of Computer Science, University of Texas at Austin, Technical Report TR-12-07 (May 2012). see http://apps.cs.utexas.edu/tech_reports/reports/tr/TR-2075.pdf
67. Moore, J.S.: (Accessed: October, 2015.) Mechanized Operational Semantics. Lectures in the Marktoberdorf Summer School (August 5-16, 2008). See http://www.cs.utexas.edu/users/moore/publications/talks/marktoberdorf-08/index.html
68. Boyer, R.S., Moore, J.S.: Single-threaded Objects in ACL2. In: Krishnamurthy, S., Ramakrishnan, C.R. (eds.) Practical Aspects of Declarative Languages (PADL), vol. 2257, pp. 9–27. Springer, LNCS (2002)
69. Hunt Jr., W.A., Kaufmann, M.: A formal model of a large memory that supports efficient execution. In: Proceedings of the 12th International Conference on Formal Methods in Computer-Aided Design (FMCAD 2012, Cambrige, UK, October 22–25) (2012). http://www.cs.utexas.edu/~hunt/FMCAD/FMCAD12/014.pdf
70. ACL2 Feature: Abstract Stobjs (Accessed: October, 2015). http://www.cs.utexas.edu/users/moore/acl2/manuals/current/manual/index.html?topic=ACL2____DEFABSSTOBJ
71. Kaufmann, M., Sumners, R.: Efficient rewriting of operations on finite structures in ACL2. In: ETAPS 2002: European joint conference on theory and practice of software. Satellite workshop, pp. 141–150 (2002)
72. Greve, D.: Scalable normalization for heap manipulating functions. In: ACL2 Workshop 2007 (2007). http://www.cs.uwyo.edu/~ruben/acl2-07/uploads/Main/017.pdf
73. Goel, S., Hunt Jr., W.A., Kaufmann, M., Ghosh, S.: Simulation and formal verification of x86 machine-code programs that make system calls. In: Proceedings of the 14th Conference on Formal Methods in Computer-Aided Design (FMCAD'14), pp. 18:91–98 (2014). http://dl.acm.org/citation.cfm?id=2682923.2682944
74. Bitops (Accessed: October, 2015) An ACL2 Library for Reasoning about Bit-Vector Arithmetic. See http://www.cs.utexas.edu/users/moore/acl2/manuals/current/manual/?topic=ACL2____BITOPS
75. Rager D.L.: (Accessed: October, 2015.) Maintaining community [in] sanity with Jenkins and Github. ACL2 Workshop 2015, Rump Session Talk. See https://www.cs.utexas.

edu/users/moore/acl2/workshop-2015/rump-session-abstracts.html#rager and http://leeroy.defthm.com/
76. Matz, M., Hubicka, J., Jaeger, A., Mitchell, M.: Chapter 4: Object Files in System V Application Binary Interface. AMD64 Architecture Processor Supplement, Draft v0 99 (2005)
77. Mach-O File Format (Accessed: October, 2015.) OS X ABI Mach-O File Format Reference. Mac Developer Library. https://developer.apple.com/library/mac/documentation/DeveloperTools/Conceptual/MachORuntime/index.html
78. Swords, S., Davis, J.: Bit-blasting ACL2 theorems. In: Proceedings 10th International Workshop on the ACL2 Theorem Prover and its Applications, ACL2 2011, Austin, Texas, USA, November 3–4, 2011, pp. 84–102 (2011). doi:10.4204/EPTCS.70.7
79. Anderson, S.: (Accessed: 2015) Bit Twiddling Hacks. See http://graphics.stanford.edu/~seander/bithacks.html
80. Goel, S., Hunt Jr., W.A.: Automated code proofs on a formal model of the x86. In: Verified Software: Theories, Tools, Experiments (VSTTE'13). Lecture Notes in Computer Science, vol. 8164, pp. 222–241. Springer, Berlin, Heidelberg (2014). doi:10.1007/978-3-642-54108-7_12
81. Moore, J.S.: Stateman: using metafunctions to manage large terms representing machine states. In: Kaufmann, M., Rager, D.L. (eds.) Proceedings Thirteenth International Workshop on the ACL2 Theorem Prover and Its Applications, Austin, Texas, USA, 1-2 October 2015, Open Publishing Association, Electronic Proceedings in Theoretical Computer Science, vol. 192, pp. 93–109 (2015). doi:10.4204/EPTCS.192.8
82. Turing, A.M.: Checking a Large Routine, pp. 67–69 (1949). http://www.turingarchive.org/browse.php/B/8
83. Hunt Jr., W.A.: Microprocessor design verification. J. Autom. Reason. **5**(4), 429–460 (1989). http://www.cs.utexas.edu/~boyer/ftp/cli-reports/048.pdf
84. Moore, J.S.: Piton: A Mechanically Verified Assembly-Level Language. Automated Reasoning Series. Kluwer Academic Publishers (1996)
85. Young, William D.: A mechanically verified code generator. J. Autom. Reason. **5**(4), 493–518 (1989)
86. Boyer, R.S., Yu, Y.: Automated proofs of object code for a widely used microprocessor. J. ACM **43**(1), 166–192 (1996). http://dl.acm.org/citation.cfm?id=227603
87. Bryant, R.E., O'Hallaron, D.R.: Chapter 4: Processor Architecture, of Computer Systems: A Programmer's Perspective. Prentice-Hall (2003)
88. Goel, S., Hunt Jr., W.A., Kaufmann, M., Krug, R.: (Accessed: 2015) y86 Specifications in the ACL2 Community Books. See https://github.com/acl2/acl2/tree/master/books/models/y86
89. Greve, D.A.: Symbolic simulation of the JEM1 microprocessor. In: Gopalakrishnan, G., Windley, P., (eds.) Formal Methods in Computer-Aided Design. Lecture Notes in Computer Science, vol. 1522, pp. 321–333. Springer, Berlin, Heidelberg (1998). doi:10.1007/3-540-49519-3_21
90. Owre, S., Rushby, J.M., Shankar, N.: PVS: a prototype verification system. In: Kapur, D. (ed.) 11th International Conference on Automated Deduction (CADE). Lecture Notes in Artificial Intelligence, vol. 607, pp. 748–752. Springer, Saratoga, NY (1992)
91. Greve, D., Wilding, M., Hardin, D.: High-speed, analyzable simulators. In: Computer-Aided Reasoning, pp. 113–135. Springer (2000)
92. Wilding, M., Greve, D., Hardin, D.: Efficient simulation of formal processor models. Form. Methods Syst. Des. **18**(3), 233–248 (2001)
93. Fox, A.: Formal specification and verification of ARM6. In: Theorem Proving in Higher Order Logics, pp. 25–40. Springer (2003)
94. Alglave, J., Fox, A., Ishtiaq, S., Myreen, M.O., Sarkar, S., Sewell, P., Nardelli, F.Z.: The semantics of power and ARM multiprocessor machine code. In: Proceedings of DAMP 2009: the 4th Workshop on Declarative Aspects of Multicore Programming, ACM, New York, NY, USA, 553091 (2009)
95. Sewell, P., Sarkar, S., Owens, S., Zappa Nardelli, F., Myreen, M.O.: x86-TSO: a rigorous and usable programmer's model for x86 multiprocessors. Commun. ACM **53**(7), 89–97 (2010) (Research Highlights)

96. Owens, S., Sarkar, S., Sewell, P.: A better x86 memory model: x86-TSO. In: Proceedings of TPHOLs 2009: Theorem Proving in Higher Order Logics, LNCS 5674, pp. 391–407 (2009)
97. Sarkar, S., Sewell, P., Zappa Nardelli, F., Owens, S., Ridge, T., Braibant, T., Myreen, M., Alglave, J.: The semantics of x86-CC multiprocessor machine code. In: Proceedings of POPL 2009: the 36th annual ACM SIGPLAN-SIGACT symposium on Principles of Programming Languages, pp. 379–391 (2009). doi:10.1145/1594834.1480929
98. Gordon, M.J.C., Melham, T.F. (eds.): Introduction to HOL: A Theorem-Proving Environment for Higher-Order Logic. Cambridge University Press, Cambridge (1993)
99. Fox, A., Myreen, M.O.: A trustworthy monadic formalization of the ARMv7 instruction set architecture. In: Kaufmann, M., Paulson, L.C., (eds.) Interactive Theorem Proving, Lecture Notes in Computer Science, vol. 6172, pp. 243–258. Springer, Berlin (2010). doi:10.1007/978-3-642-14052-5_18
100. Myreen, M.O., Gordon, M., Slind, K.: Machine-code verification for multiple architectures - an application of decompilation into logic. In: Formal Methods in Computer-Aided Design, 2008. FMCAD '08, pp. 1–8 (2008). doi:10.1109/FMCAD.2008.ECP.24, http://www.cl.cam.ac.uk/~mom22/decomp.pdf
101. Klein, G., Elphinstone, K., Heiser, G., Andronick, J., Cock, D., Derrin, P., Elkaduwe, D., Engelhardt, K., Kolanski, R., Norrish, M., et al.: seL4: formal verification of an OS kernel. In: Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles, pp. 207–220. ACM (2009). http://www.sigops.org/sosp/sosp09/papers/klein-sosp09.pdf
102. sel4: General Dynamics C4 Systems (Accessed: October, 2015). http://sel4.systems/
103. Morrisett, G.: Scalable formal machine models. In: Proceedings of the Second International Conference on Certified Programs and Proofs, pp. 1–3. Springer, Berlin, Heidelberg, CPP'12 (2012). doi10.1007/978-3-642-35308-6_1
104. Coq Proof Assistant (Accessed: October, 2015). http://coq.inria.fr/
105. Feng, X., Shao, Z., Guo, Y., Dong, Y.: Certifying low-level programs with hardware interrupts and preemptive threads. J. Autom. Reason. **42**(2), 301–347 (2009). http://flint.cs.yale.edu/flint/publications/aimjar.pdf
106. Shao, Z.: Clean-slate development of certified OS kernels. In: Proceedings of the 2015 Workshop on Certified Programs and Proofs, pp. 95–96. ACM, New York, NY, USA, CPP '15 (2015). doi:10.1145/2676724.2693180
107. Fox, A.: Directions in ISA Specification. Interactive Theorem Proving (ITP), pp. 338–344 (2012). https://www.cl.cam.ac.uk/~acjf3/papers/itp12.pdf
108. Degenbaev, U.: Formal Specification of the x86 Instruction Set Architecture. PhD thesis, Universität des Saarlandes (2012). http://rg-master.cs.uni-sb.de/publikationen/UD11.pdf
109. Shi, X.: Certification of an instruction set simulator. PhD thesis, Université de Grenoble (2013)
110. Leroy, X.: Formal verification of a realistic compiler. Commun. ACM **52**(7), 107–115 (2009). http://gallium.inria.fr/~xleroy/publi/compcert-CACM.pdf
111. Bjørner, D.: A ProCoS project description. International Conference on AI and Robotics, North Holland (1989)
112. Bowen, J.P.: A ProCoS II project description: ESPRIT Basic Research project 7071. Bull. Eur. Assoc. Theor. Comput. Sci. (EATCS) **50**, 128–137 (1993)